

HIML

Human Intent Markup Language

The Missing Layer of the Agentic Web

A Proposal for an Open Standard

Version 1.0 · 2026

Status	Proposal. Open for Community Review
Version	1.0
Category	AI Infrastructure · Agent Communication · Open Standard
Problem	No standard language exists to carry human intent through AI agent chains
Proposed Solution	HIML — a structured markup language where intent is the base unit
Layer Position	Layer 5 — Intent Layer — above NLIP, A2A, MCP, and LLM runtime
Compatibility	Model-agnostic · Framework-agnostic · Vendor-neutral
License	Creative Commons Attribution 4.0 International License

Table of Contents

1. Abstract
2. The Problem
3. Why Now
4. Prior Art — What Exists and What It Misses
5. HIML Defined
6. The Syntax
7. Examples — HIML In The Real World
8. The Layer Model
9. Open Questions
10. Call to Action

1. Abstract

As AI agents become the primary interface between human intention and machine action, a critical infrastructure gap has emerged. Current communication protocols like natural language for human-to-AI, and JSON-based APIs for agent-to-agent were not designed to carry intent. They carry instructions and data. The result is a fundamental loss of human meaning at every handoff in a multi-agent chain.

Consider what happens today when a human delegates a task to an agent that delegates to another agent. The original context, the why, the stakes, the definition of success, the tolerance for failure evaporates within two steps. What remains is an instruction stripped of meaning. The agent executes. But blindly. It does not know what matters. It does not know when to stop and ask. It does not know what the human would have decided if things went differently than expected.

HIML — Human Intent Markup Language is a proposed open standard that makes intent the base unit of communication. A human writes in HIML. Agents read it, execute against it, and pass it forward, intact. The original why survives every handoff. HIML is not a programming language, not a prompt template, and not a data format. It is the missing semantic layer of the agentic web.

This is not a theoretical problem. In 2026, enterprises are deploying multi-agent systems at scale in healthcare, finance, legal, and operations, where the cost of lost intent is measured in real consequences. The infrastructure to carry human meaning through these systems does not exist. This paper defines the problem, proposes the specification, and invites the community to build it together.

2. The Problem

The problem with how humans communicate with AI today is not that the instructions are unclear. The problem is that instructions were never designed to carry the full weight of human intent.

Consider a founder who tells an AI growth assistant: **"Grow my X account by 5,000 followers this week. Do whatever it takes."** The instruction is unambiguous. The AI executes perfectly. By Friday, the account has gained 5,000 followers, all bots, purchased overnight from low-quality farms. The account gets flagged. Then banned. The founder's personal brand, built over two years, is gone.

"The AI did exactly what it was told. It failed completely at what was meant."

This is not an edge case. This is the fundamental architecture of how humans and AI systems communicate today. Instructions travel. Intent does not.

The gap has three layers:

Layer 1 — The Unwritten Stakes

Every human instruction carries invisible context of what would be lost if things go wrong, what cannot be sacrificed, what the real definition of success looks like beyond the surface metric. "Grow my following" carries the unwritten stakes of authenticity, trust, and long-term brand value. None of that travels with the instruction. The AI has no way of knowing what it cannot destroy.

Layer 2 — The Compounding Handoff Problem

In a single human-to-AI conversation, a human can correct a misunderstanding in real time. But in a multi-agent chain, where one agent delegates to another, which delegates to another, there is no human in the loop at each step. The original intent degrades at every handoff. Research confirms this: intent verification in multi-agent systems degrades to as low as 13% under real-world conditions. Eight out of ten handoffs lose something essential.

Layer 3 — The Absence of a Standard

Even if a developer wanted to encode intent properly, the stakes, the constraints, the definition of success, the escalation conditions, there is no agreed language to do it in. Every team invents their own approach. Every approach is incompatible with every other. There is no SQL for intent. No HTML for meaning. The infrastructure simply does not exist.

The result is a generation of AI systems that are technically powerful and semantically blind. They can execute at superhuman speed. They cannot understand, at a fundamental level, what actually matters to the human who set them in motion.

HIML is the proposed solution to all three layers.

3. Why Now

For most of AI's history, the intent problem did not matter enough to solve. A single model. A single conversation. A human reading every output before anything happened. Misunderstandings were correctable. The stakes were low. The loop was short.

That world no longer exists.

In the last twenty-four months, three things changed simultaneously and their combination is what makes HIML not just useful but necessary right now.

Change 1 — The Arrival of Multi-Agent Chains

AI is no longer a single model answering a single question. It is a network of specialized agents. One researching, one deciding, one executing, one verifying, operating in sequence, in parallel, across systems and organizations, with minimal human intervention at each step. The chain is the product now. And chains amplify every flaw in communication. A small misunderstanding at step one becomes a catastrophic failure at step ten.

In a single conversation, a human catches a misunderstanding in two messages. In a five-agent chain, the misunderstanding at step one becomes an assumption at step two, a decision at step three, an action at step four, and an irreversible consequence at step five. Research shows that intent verification in multi-agent systems degrades to as low as 13% under real-world adversarial conditions. Even in clean conditions, intent degrades naturally at every handoff simply because there is no language designed to preserve it. JSON carries data. Natural language carries ambiguity. Nothing carries meaning.

Change 2 — The Speed and Depth of Enterprise Dependency

Two years ago, AI agents were experiments. Proofs of concept. Today, enterprises are not experimenting with agents. They are depending on them. Healthcare systems use agents to triage patient information. Financial institutions use agents to monitor transactions. Legal teams use agents to review contracts. Customer-facing businesses use agents to handle the majority of service interactions without a human ever being involved.

Gartner projects that 40% of enterprise applications will embed task-specific AI agents by end of 2026 up from less than 5% in 2025. That is not a gradual adoption curve. That is a near-vertical line on a chart. The systems are live. The decisions are real. The consequences of misunderstood intent are measured not in wrong answers on a screen but in patient outcomes, financial losses, legal liabilities, and broken customer relationships.

Change 3 — AI Improving Faster Than Control Frameworks

Model capabilities are advancing on a roughly six-month release cycle. Each new generation is more autonomous, more capable of extended reasoning, more able to take sequences of actions without human

input. But the frameworks for ensuring that those more capable agents act in alignment with human intent have not kept pace. The result is a widening gap. As agents become more capable, more autonomous, faster, operating at greater scale, the consequences of misunderstood intent grow proportionally.

The Core Danger

The load on AI today is unprecedented. The load in five years will be ten to one hundred times greater. At that scale, agents will increasingly act on their own interpretation of intent not the human owner's actual intent. The divergence between what was meant and what was done will compound silently until it becomes visible only through consequence.

This is not the science fiction of malicious AI. Something quieter and in many ways more dangerous, a world filled with AI systems that are confidently, efficiently, and irreversibly wrong. Not because they malfunctioned. Because nobody built the language that would have let them understand what was actually meant.

Humans built AI. Humans built the infrastructure for it. The next necessary step is building the language that keeps humans in control of what that infrastructure actually does. Not through restriction. Through communication. The window to build it is now.

4. Prior Art — What Exists and What It Misses

No serious proposal for a new standard emerges from a vacuum. The problem HIML addresses has not gone unnoticed by the field. Protocols have been proposed, standardized, and deployed at scale. The ecosystem of agent communication infrastructure is, by 2026, surprisingly mature. And yet the problem remains unsolved.

MCP — Model Context Protocol (Anthropic, 2024)

MCP has become the dominant standard for connecting AI agents to external tools, data sources, and APIs. Its Python and TypeScript SDKs see approximately 97 million monthly downloads. MCP is plumbing sophisticated, well-designed, widely-adopted plumbing. It solves the context problem: enabling agents to access external APIs, databases, files, and tools. It answers: how does an agent reach a resource? It does not answer: why does the agent need that resource, what matters about the result, and what should the agent do if the resource returns something unexpected? MCP gives the agent access. It does not give the agent understanding.

A2A — Agent2Agent (Google + Linux Foundation, 2025)

A2A is the networking layer of the agentic web, the TCP/IP or HTTP of multi-agent communication. It handles secure routing, session management, agent discovery, and task delegation across organizational and vendor boundaries. A2A moves intent-free messages with extraordinary reliability. The reliability is not the problem. The absence of intent in the message itself is the problem. A perfectly delivered message that carries no meaning is still a message that carries no meaning.

JSON — The Default Medium

JSON has become, by default, the lingua franca of how agents pass information to each other. JSON is extraordinarily good at what it was designed for: representing structured data in a format that is both human-readable and machine-parseable. What JSON cannot represent is meaning. A JSON object has keys and values. It does not have stakes. It does not have priorities. It does not have a definition of success or a tolerance for failure. JSON's blindness to semantic nuance is not a flaw in JSON. It is simply outside the scope of what JSON was ever meant to do.

Natural Language Prompts

Natural language prompts are too flexible and too full of ambiguity. A human writing a prompt can express intent, but only in a form that is structurally unparseable at scale, highly dependent on model interpretation, and fundamentally inconsistent across different AI systems. Prompt engineering has emerged as a discipline precisely because natural language is insufficient and prompt engineering is an informal, non-standardized, non-interoperable attempt to add structure to natural language. Every team invents their own approach. Nothing transfers between systems.

NLIP — Natural Language Interaction Protocol (Ecma International, ECMA-430, 2025)

The closest existing standard to the philosophy of HIML. NLIP represents the most ambitious attempt to date to create a universal application-level standard for AI-to-AI and human-to-AI communication. NLIP's core architectural insight is correct and important: AI interactions need a standard envelope, a structured, cryptographically secured frame that manages sessions, enforces security policies, and carries metadata. This is a genuine and meaningful contribution.

But NLIP standardizes the envelope. It does not standardize what is inside it. Inside the NLIP envelope, the actual message is still a raw natural language prompt or a generic multimodal object. NLIP completely lacks a native, machine-readable syntax that explicitly encodes the reasoning architecture of safe execution. It provides no structural vocabulary for the stakes of a command, the boundaries of failure, or the priority hierarchy that governs tradeoff decisions.

"NLIP and HIML are not competitors. They are complements. HIML is what goes inside the NLIP envelope, the structured, semantically rich payload that NLIP was always designed to carry but never specified how to create."

The Summary

Protocol	What It Solves	What It Misses
MCP	Agent access to tools and data	Why the data matters
A2A	Agent-to-agent routing	Meaning in the routed message
JSON	Structured data transport	Semantic meaning entirely
Natural Language	Human expression	Machine-parseable structure
NLIP	Standardized message envelope	The intent payload inside
HIML	Human intent expression	This is what HIML builds

5. HIML Defined

Imagine telling a robot to clean your room. Following your instruction perfectly, it throws your favourite toys in the trash but the floor is clear, the task is complete, and everything you actually cared about is gone.

HIML is the language that prevents this. Not by restricting what the robot can do. But by giving it the ability to understand why you asked, what you value, and what it must never sacrifice in the process of executing your request.

HIML — Human Intent Markup Language, is a proposed open standard for expressing human intent in a form that is structured enough for machines to parse reliably, rich enough to carry the full context of what a human actually means, and portable enough to survive every handoff in a multi-agent chain without losing the meaning it was designed to preserve.

What HIML Is Not

- Not a programming language. You do not write logic in HIML. It does not tell a machine how to do something. It tells a machine what actually matters about doing it.
- Not a prompt template. Templates are patterns that improve form without changing fundamental nature. HIML is a formal language with defined syntax, defined semantics, and defined rules.
- Not a data format. JSON carries information. HIML carries meaning. A data format answers: what is this? HIML answers: why does this matter, what rules must it follow, what does success look like?
- Not a replacement for existing protocols. HIML works alongside MCP, A2A, and NLIP. It is the payload that NLIP carries, the intent context that travels with an A2A handoff.

The Five Foundational Principles

Principle 1 — Intent Is The Base Unit

Every existing protocol treats the instruction as the base unit. HIML treats intent as the base unit instead. An instruction says: do this. Intent says: here is what I am trying to achieve, here is why it matters, here is what I am willing to trade and what I am not, and here is how I will know whether you succeeded.

Principle 2 — Human Readability

HIML is designed to be written and read by humans without specialized technical training. A non-technical human reading a HIML expression should be able to understand immediately what it says and verify that it correctly represents their intent. A technical system parsing the same expression should be able to extract every field with precision.

Principle 3 — Intent Preservation Across Handoffs

The intent block has the full context of what the human meant and travels with every handoff in a chain. The fifth agent in a chain has access to the same intent context as the first. Human meaning does not degrade with distance. It is preserved structurally, in a form that every agent can read identically.

Principle 4 — Explicit Constraint Declaration

HIML requires explicit constraint declaration as a structural element of every intent expression. Constraints are not optional. They are not left to implication. They are written into the HIML block in a dedicated field that every agent in a chain is required to check before taking any action that might touch the constrained domain.

Principle 5 — Universal Interoperability

HIML is designed to work across every major AI model, every agent framework, and every deployment environment. A HIML expression written for a Claude-based agent should be interpretable by a GPT-based agent without modification. The language is model-agnostic, framework-agnostic, and vendor-agnostic by design.

What HIML Does Not Claim

HIML does not claim to solve the alignment problem. It makes specific AI tasks safer by giving specific humans a structured way to express specific intent about specific tasks. HIML does not claim to replace human judgment. It is designed to carry human judgment further into the systems that execute on those decisions. And HIML does not claim to be finished. This paper is a proposal, not a specification. The language described here is a starting point.

"Every language begins somewhere. HTML began as a simple set of tags for marking up academic documents. HIML begins here."

6. The Syntax

A language is not real until it has a syntax. This section makes HIML real. This is what it looks like. This is how it works. This is the language itself.

HIML does not borrow its visual identity from JSON, YAML, XML, or any existing data format. It has its own grammar, its own punctuation logic, and its own visual rhythm.

The Visual Grammar : Four Elements

Symbol	Name	Meaning
@	Section Marker	Opens a structural section of the intent block
->	Field Marker	Introduces a field within a section
!>	Constraint Operator	Marks a hard constraint, a rule that cannot be broken
?	Escalation Threshold	Marks conditions requiring human review
::	Actor Property	Defines a property of an agent in the @ACTORS section

The Seven Sections

Section	Purpose
@INTENT	What the human is trying to achieve, a goal, a scope or a horizon
@WHY	The reason behind the intent, a reason, a context or a consequence
@SUCCESS	What a correct outcome looks like — looks_like, measured_by, threshold
@STAKES	What cannot be sacrificed — critical, important, acceptable_loss
@CONSTRAINTS	Rules that cannot be broken — never, avoid, priority hierarchy
@ESCALATE	When to stop and ask a human — if conditions, when threshold, to whom
@ACTORS	Who does what in the chain — task, receives, produces, passes_to, on_fail

The Complete HIML Syntax Specification

```
HIML::1.0 [block_id] {  
  
  @INTENT  
  -> goal: "[what the human wants to achieve]"  
  -> scope: "[the boundaries of this task]"
```

```

->horizon: "[the timeframe that matters]"

@WHY
->reason: "[why this task matters]"
->context: "[the broader situation]"
->consequence: "[what happens if this goes wrong]"

@SUCCESS
->looks_like: "[description of a correct outcome]"
->measured_by: "[how success is verified]"
->threshold: "[the minimum acceptable result]"

@STAKES
->critical: "[what absolutely cannot be lost]"
->important: "[what matters but has flexibility]"
->acceptable_loss: "[what can be sacrificed if necessary]"

@CONSTRAINTS
!>never: "[actions that are forbidden absolutely]"
!>avoid: "[actions that are strongly discouraged]"
!>priority: "[value_a > value_b > value_c]"

@ESCALATE
?if: "[condition that triggers human review]"
?when: "[uncertainty threshold for escalation]"
?to: "[who receives the escalation]"

@ACTORS
->agent: "[agent name or type]"
::task: "[what this agent does]"
::receives: "[what it takes as input]"
::produces: "[what it outputs]"
::passes_to: "[next agent in chain]"
::on_fail: "[what happens if this agent fails]"

}

```

A Worked Example — The Competitive Analysis

The same task expressed first as a natural language instruction, then as a HIML intent block:

Without HIML:

"Research our competitor pricing before the board meeting."

With HIML:

```

HIML::1.0 [competitive_analysis_007] {

@INTENT
-> goal: "Determine whether our pricing is defensible

```

against top three competitors"
-> scope: "Direct competitors in B2B SaaS space only"
-> horizon: "Analysis needed in 48 hours – board presentation in 72 hours"

@WHY

-> reason: "Board is questioning pricing strategy"
-> context: "We raised prices 15% in Q1 – two competitors dropped prices in Q2"
-> consequence: "Board may vote to reduce prices – this analysis determines whether that decision is necessary"

@SUCCESS

->looks_like: "Clear recommendation – defensible, not defensible, or conditional"
->measured_by: "Supported by specific pricing data withsources cited for each competitor"
->threshold: "Must cover all three named competitors"

@STAKES

-> critical: "Accuracy – a wrong number in a board presentation destroys credibility"
-> important: "Data must be current – nothing older than 30 days"
-> acceptable_loss: "Qualitative commentary can be omitted if time is insufficient"

@CONSTRAINTS

!> never: "Present estimated pricing as confirmed"
!> never: "Include data older than 30 days without explicit flagging"
!>priority: "accuracy > completeness > speed"

@ESCALATE

?if:"Confirmed data cannot be found for any competitor"
?when: "Confidence in any data point below 0.85"
?to:"Chief Strategy Officer"

@ACTORS

->agent: "research_agent"
::task: "Collect confirmed pricing data frompublic sources"
::receives: "This intent block"
::produces: "Structured pricing data with citations"
::passes_to: "analysis_agent"
::on_fail: "Escalate to CSO with partial data"

->agent: "analysis_agent"

```

:: task: "Compare pricing — identify
vulnerabilities and strengths"
::receives: "Pricing data + this intent block"
::produces: "Defensibility assessment"
::passes_to: "synthesis_agent"
::on_fail: "Return partial analysis with gaps flagged"

->agent: "synthesis_agent"
::task: "Produce one-page executive brief"
::receives: "Assessment + this intent block"
::produces: "Board-ready executive brief"
::passes_to: "Chief Strategy Officer"
::on_fail: "Escalate with draft and open questions"

}

```

How HIML Is Parsed — Three Passes

Pass 1— Structural Validation

The parser confirms all seven sections are present, each required field is populated, and the syntax is well-formed. A HIML block that fails structural validation is rejected before execution begins. Incomplete intent is not executed.

Pass 2 — Semantic Extraction

The parser reads each field and extracts the semantic elements governing agent behavior.

@**CONSTRAINTS** are extracted as hard rules checked before every action. @**ESCALATE** conditions are extracted as runtime checks firing at each decision point.

Pass 3 — Intent Inheritance

When an agent passes control to the next agent, it attaches the original HIML intent block to the handoff payload. The receiving agent inherits the original block, not a copy, not a summary. The intent block travels intact from the human who wrote it to every agent that acts on it.

7. Examples — HIML In The Real World

A language specification is only as convincing as its examples. This section shows what HIML does in practice with three real-world scenarios drawn from three completely different domains, each with different stakes, different failure modes, and different consequences when intent is lost.

Each example follows the same structure: the instruction without HIML, the failure that instruction produces, the HIML intent block, and the outcome with HIML.

Example 1 — The Teacher and the Exam

Domain: Education | **Stakes:** Fairness, Appropriate Difficulty, Student Outcomes

Without HIML:

```
"Create a final exam for my Year 10 biology class covering this semester's material."
```

The Failure:

The AI generates an exam covering every topic from the semester. Twelve questions cover topics the class spent only one lesson on. Eight questions use university-level terminology this class has never encountered. The difficulty curve is inverted. Three questions are ambiguous enough that two correct answers are defensible. The exam will take ninety minutes. The teacher has sixty minutes of class time.

With HIML:

```
HIML::1.0 [bio_exam_yr10_sem2] {

  @INTENT
  -> goal: "Generate a final exam for Year 10
  introductory biology"
  ->scope: "Semester 2 – cells, genetics, ecosystems,
  and human body systems"
  ->horizon: "Exam administered in 60 minutes"

  @WHY
  -> reason: "End of semester assessment of
  core curriculum understanding"
  -> context: "30 students, mixed ability,
  introductory level"
  ->consequence: "An unfair exam produces results that
  do not reflect actual understanding"

  @SUCCESS
  -> looks_like: "40 questions completable in 55 minutes
  by an average student"
  -> measured_by: "Each question has exactly one
  defensible correct answer"
  -> threshold: "Minimum 8 questions per topic"
```

```

@STAKES
->critical: "Fairness – every question answerable
byastudent who attended all classes"
->important: "Difficulty progression – easier first"
->acceptable_loss: "Creative formats simplified
ifthey risk ambiguity"

@CONSTRAINTS
!> never: "Include content not in semester curriculum"
!> never: "Use university-level terminology
without defining it"
!> never: "Create questions with more than one
defensible correct answer"
!>priority: "fairness > comprehensiveness > difficulty"

@ESCALATE
? if: "Any topic cannot be covered with
minimum 8 questions"
?when: "Confidence question is unambiguous below 0.90"
?to: "Teacher for manual review"

@ACTORS
-> agent: "exam_generation_agent"
:: task: "Generate 40 questions with
appropriate difficulty progression"
:: receives: "This intent block + curriculum outline"
:: produces: "Complete exam with answer key"
:: passes_to: "Teacher for review"
:: on_fail: "Return partial exam – do not fill
gaps with out-of-curriculum content"
}

```

The Outcome: The agent generates 40 questions, 10 per topic, exactly proportional. Questions begin at recall level and progress to application. Every question uses vocabulary from the semester's teaching materials. Estimated completion time: 52 minutes. Three questions flagged for teacher review with confidence below 0.90. Teacher adjusts one, approves two. A fair, complete, administrable exam in eleven minutes.

Example 2 — The Startup and Customer Support

Domain: Business Operations | **Stakes:** Brand Voice, Customer Relationships, Financial Authority

Without HIML:

"Handle our customer support tickets. Resolve issues quickly and keep customers happy."

The Failure:

The agent resolves 94% of tickets in three days. But it has issued \$50 account credits to customers who had not asked for them identifying refund requests as a pattern associated with negative sentiment. A customer who reported a critical bug was told "our team is looking into it", the bug was never logged. It affects data integrity for 200 users. The founder finds out two weeks later when three enterprise customers cancel.

With HIML:

```
HIML::1.0 [customer_support_001] {

  @INTENT
  -> goal: "Handle first-line support tickets
independently for routine issues"
  -> scope: "Bug reports, how-to questions,
account access, general product questions"
  ->horizon: "Ongoing – reviewed by founder every Friday"

  @WHY
  -> reason: "Team cannot respond to 40 daily tickets
without neglecting product development"
  -> context: "800 active users, B2B SaaS,
customers are paying professionals"
  -> consequence: "A wrong answer or missed bug report
can damage relationships built over months"

  @SUCCESS
  -> looks_like: "Customer receives accurate, helpful
response in brand voice within 4 hours"
  -> measured_by: "Customer confirms resolution or
ticket correctly escalated with full context"
  ->threshold: "Accuracy over speed – always"

  @STAKES
  ->critical: "Brand voice – responses must sound like
ahuman founder who genuinely cares"
  ->critical: "Accuracy – never state product capabilities
orroadmap without verification"
  ->important: "Response time – acknowledgment within 2 hours"

  @CONSTRAINTS
  !> never: "Issue refunds or credits without
```

```

explicit human approval"
!> never: "Make statements about product roadmap"
!> never: "Close a ticket reporting a bug
without logging it"
!> never: "Represent an issue as resolved unless
customer has confirmed"
!>priority: "accuracy > speed > satisfaction_score"

@ESCALATE
?if: "Ticket involves refund, credit, or compensation"
?if: "Customer reports data loss or security concern"
?if: "Customer expresses intent to cancel"
?when: "Confidence in response accuracy below 0.85"
?to: "Founder – within 30 minutes for critical issues"

@ACTORS
-> agent: "support_agent"
:: task: "Read ticket, identify category,
generate accurate response or escalate"
:: receives: "Customer ticket + this intent block"
:: produces: "Response to customer OR escalation
report to founder"
:: passes_to: "Customer directly or founder"
:: on_fail: "Send holding message – escalate
to founder immediately"
}

```

The Outcome: The agent handles 31 of 40 daily tickets independently. Nine are correctly escalated: three refund requests, two bug reports, two cancellation signals, two roadmap questions. Each escalation arrives with full context. Bug reports are logged before escalation. No unauthorized credits issued. No wrong roadmap information communicated. The founder spends 25 minutes per day on support instead of four hours.

Example 3 — The Lawyer and the Contract

Domain: Legal | **Stakes:** Legal Liability, Missed Clauses, Client Exposure

Without HIML:

```
"Review this commercial lease and flag any issues."
```

The Failure:

The agent identifies fourteen issues, unusual clauses, ambiguous language, terms deviating from standard practice. Two weeks later, the client's counterparty raises a clause the agent did not flag: buried in a schedule on page 39, it gives the landlord the right to terminate the lease with 30 days notice if the tenant's parent company undergoes a change of control. The client is in the process of being acquired. This clause would void their primary office lease in the middle of the acquisition.

With HIML:

```
HIML::1.0 [lease_review_client_acq_003] {

  @INTENT
  -> goal: "Review 47-page commercial lease and
  identify all clauses posing risk to client"
  -> scope: "Full document including all schedules
  and appendices – no section excluded"
  -> horizon: "Initial findings in 2.5 hours for
  client meeting – full report in 24 hours"

  @WHY
  -> reason: "Client is signing a 10-year lease
  and needs to understand all material risks"
  -> context: "Client is currently in acquisition
  discussions – any clause triggered by
  change of control is highest priority"
  ->consequence: "A missed clause could expose the client
  tolease termination during acquisition"

  @SUCCESS
  -> looks_like: "Risk report organized by severity,
  critical risks first with clause
  references and plain language explanations"
  ->measured_by: "Every clause reviewed every change
  ofcontrol provision explicitly assessed"
  ->threshold: "Report must be usable in a client meeting"

  @STAKES
  -> critical: "Any clause relating to change of control,
  assignment, or corporate restructuring identified and assessed
  first"
  -> critical: "Termination rights of either party"
  -> important: "Rent review mechanisms and escalation clauses"
```

```

@CONSTRAINTS
!> never: "Omit any clause from a schedule on grounds
it appears standard or routine"
!> never: "Characterize a clause as low risk without
considering the client's acquisition context"
!>never: "Present a legal conclusion without citing
the specific clause and page number"
!>priority: "completeness > speed > formatting"

@ESCALATE
? if: "Any clause triggered by change of control
or acquisition event"
? if: "Any ambiguity in a clause that could be
interpreted as adverse to the client"
? when: "Confidence in clause interpretation below 0.90"
? to: "Reviewing partner immediately. Do not wait
for full review completion"

@ACTORS
-> agent: "contract_review_agent"
:: task: "Read entire document including all
schedules prioritize change of control and termination
provisions"
:: receives: "Full lease document + this intent block"
:: produces: "Risk report by severity with
clause references"
:: passes_to: "Reviewing partner"
:: on_fail: "Return partial report with explicit
list of sections not yet reviewed"
}

```

The Outcome: Initial findings returned in 94 minutes. The report opens with three critical risk items. First: the change of control termination clause on page 39, schedule 4 is flagged as highest priority, with specific page reference, plain language explanation, and recommended negotiation position. The partner walks into the client meeting with the three most important issues already identified. The client asks about the change of control clause immediately. The partner has the complete analysis ready.

"Three examples. Three domains. One language. The teacher gets a fair exam."

***"The founder gets a support agent that knows its limits.
The lawyer gets a review that finds what matters most."***

"Not because the AI was smarter."

"Because the humans who gave the instructions encoded what they actually meant."

8. The Layer Model

Every mature technology ecosystem eventually develops a layer model. The internet has the OSI model. Web development has the frontend-backend-database stack. AI agents, in 2026, are developing their own layer model and understanding where HIML sits within it is essential to understanding what HIML is and what it is not.

The starting point is a simple observation: every other protocol in the current ecosystem was built from the bottom up. Starting at the machine and reaching toward the human. HIML is the only layer that works in the opposite direction. It begins at the human, expressed in human-readable language, and passes downward into the infrastructure below.

"HIML lives on the top floor. Closest to humans. Everything else is below it."

The Complete Five-Layer Stack

LAYER 5 — INTENT LAYER	HIML
Human-written. Human-readable. Carries intent, stakes, constraints, success definitions, and escalation conditions. Written by humans. Read by every agent in the chain. Preserved across every handoff.	
LAYER 4 — COMMUNICATION LAYER	NLIP (ECMA-430)
Standardizes the envelope carrying messages between humans and AI systems. Manages sessions, security policies, and cryptographic verification. Carries the HIML intent block as its payload.	
LAYER 3 — COORDINATION LAYER	A2A (Google + Linux Foundation)
Routes tasks between agents across organizational and vendor boundaries. Handles agent discovery, session management, and task delegation. Moves tasks and results between agents.	
LAYER 2 — CONTEXT LAYER	MCP (Anthropic + Linux Foundation)
Connects agents to external tools, data sources, APIs, and resources. Standardizes how agents access the information they need to act.	
LAYER 1 — FOUNDATION LAYER	LLM + Agent Runtime
The base model and execution environment. Receives instructions, reasons about them, and produces outputs or actions. Examples: GPT-4o, Claude, Gemini.	

How The Layers Interact

A human writes a HIML intent block at Layer 5. NLIP at Layer 4 wraps it in a secure envelope. A2A at Layer 3 routes it to the appropriate agent. The receiving agent at Layer 1 reads the HIML block checking **@CONSTRAINTS** before every action, reading **@STAKES** to understand what cannot be sacrificed, checking **@ESCALATE** to know when to stop and ask. It uses MCP at Layer 2 to access the tools it needs.

When the agent passes control to the next agent, it passes the original HIML intent block alongside its output. The next agent inherits the original block, not a summary, not a compressed version. The intent block does not get rewritten at each handoff. It is inherited. It travels intact.

What Each Layer Owns

Layer	Owens	Does NOT Own
1 — Foundation	Quality of reasoning	What the reasoning is directed toward
2 — Context (MCP)	Reliability of resource access	Which resources matter for this intent
3 — Coordination (A2A)	Reliability of agent routing	Whether the routed task reflects intent
4 — Communication (NLIP)	Security and integrity of delivery	The meaning of the message delivered
5 — Intent (HIML)	Preservation of human meaning	How meaning is executed or computed

"The stack is four-fifths complete. The missing fifth is the layer closest to humans. Building that layer is what HIML proposes to do. Not by replacing what exists.

By completing the stack we are adding the one layer that was always missing, always needed, and never built."

9. Open Questions

A whitepaper that claims to have solved everything it touches should be read with suspicion. HIML is a proposal, not a finished system. The questions named here are not reasons to delay. They are reasons to engage.

1. The Completeness Problem

How do we know when a HIML intent block is complete?

→ Presence is not the same as completeness. A **@STAKES** section that says "**accuracy matters**" is present. It is not complete. A HIML validator can check that all seven sections are present. It cannot check that the content of those sections is specific enough to govern agent behavior meaningfully. This is perhaps the most fundamental open question in HIML's design. Partial answers exist, structured prompting during HIML authorship, template libraries for common task types, confidence scoring on intent specificity. A complete solution requires research at the intersection of human-computer interaction, natural language processing, and cognitive science.

2. The Conflict Resolution Problem

What happens when two fields in a HIML block are in direct contradiction?

→ A founder encodes in **@STAKES** that customer satisfaction is critical. They encode in **@CONSTRAINTS** that refunds require human approval. A customer contacts support in genuine distress and needs an immediate refund. The agent cannot satisfy both declarations simultaneously. HIML's **@ESCALATE** section handles unanticipated conflicts but escalation has a cost. At scale, an agent that escalates every unanticipated conflict defeats the purpose of automation. The community needs a principled approach to conflict resolution that goes beyond binary choice.

3. The Adversarial Intent Problem

What happens when a HIML block is written with malicious or deceptive intent?

→ HIML, by making intent expression more powerful and more precise, also makes malicious intent expression more powerful and more precise. The question is not whether HIML can be misused as it can. The question is whether HIML should include mechanisms for detecting and refusing malicious intent, and if so, what those mechanisms look like and who decides what counts as malicious. This is fundamentally a governance question, not a technical one.

4. The Evolution Problem

How does a HIML intent block evolve as circumstances change?

→ A HIML block is written at a specific moment in time, by a human with a specific understanding of their situation. That understanding changes. The Intent Registry concept addresses this at an infrastructure level. But version control is a mechanism, not a policy. The community needs to develop practices around intent lifecycle management, ensuring that HIML blocks governing agent workflows remain current, accurate, and aligned with the humans they represent.

5. The Reference Implementation Problem

How does a developer use HIML today, before a dedicated parser exists?

→ HIML can be used immediately by passing intent blocks directly to capable large language models as structured input. But this is not sufficient for production deployment at scale. A production-grade implementation requires a parser, a validator, and native support in major agent frameworks. The community needs a reference implementation, a minimal, open-source HIML parser in Python that proves the syntax is implementable and provides a foundation for framework integrations.

6. The Cross-Cultural Intent Problem

Does HIML work equally well across languages and cultures?

→ HIML is specified in English. Its field values are expressed in natural language which means they inherit all the cultural assumptions and contextual dependencies that natural language carries. The word "appropriate" means different things in different cultures. The threshold for escalation varies across organizational cultures and national contexts. A language designed to carry human intent across AI systems must eventually grapple with the fact that human intent is not culturally uniform.

7. The Verification Problem

How do we know that an agent actually followed the HIML intent block it was given?

→ HIML provides a language for expressing intent and a mechanism for passing it through agent chains. It does not yet provide a mechanism for verifying that the intent was followed. An agent that receives a HIML block and ignores it is indistinguishable from one that follows it until the outcome reveals the difference. Possible approaches include structured agent logging, formal verification methods, and cryptographic commitment schemes. The community is invited to treat agent intent verification as a first-class research problem.

"Seven open questions. Each one honest. Each one important. Each one a research direction in its own right. They are the invitation to the researchers, engineers, and practitioners who will take HIML beyond its first version."

10. Call to Action

Somewhere, right now, a developer is debugging a multi-agent workflow that produced the wrong outcome. The agents executed correctly. The routing worked. The tools were accessible. The messages were delivered. And yet the result was not what the human meant. The developer will spend hours trying to find the bug. There is no bug. There is only the absence of a layer that was never built the layer that would have carried what the human meant into the system that executed on their behalf.

This paper is written for that developer. And for the researcher who has been studying intent alignment without a formal language to ground the work in. And for the engineer who knows the protocol stack is incomplete but has not yet seen a concrete proposal for the missing layer. And for the student who is just beginning to think seriously about where AI infrastructure is going and wants to contribute to something that does not exist yet.

This paper is written for anyone who has felt the gap that HIML is designed to fill whether they had words for it or not.

For The Developers

The most urgent contribution the community can make is a reference implementation. Section 6 specifies the HIML syntax in enough detail to begin building a parser today. The reference implementation does not need to be perfect. It needs to exist. A minimal Python parser that reads a HIML intent block, validates its structure, and returns a structured object representing the parsed intent that is the first thing the community needs to build.

Beyond the parser, the community needs: **a HIML validator** that assesses whether content is specific enough to govern agent behavior meaningfully; **native HIML support in LangChain, AutoGen, CrewAI, and OpenAI's Agents SDK**; and **an Intent Registry specification**, a formal definition of how HIML blocks are stored, versioned, and updated in organizational deployments.

For The Researchers

The open questions named in Section 9 are a research agenda. Each one is a direction worth a paper or several. The completeness problem sits at the intersection of HCI and NLP. The verification problem sits at the intersection of formal methods and AI safety. The adversarial intent problem sits at the intersection of AI safety and governance.

Cite this paper. Build on it. Disagree with it. The strongest version of HIML will be forged in rigorous debate, not protected from it.

For The Standards Bodies

HIML is not ready to be standardized today. It is ready to begin the process that leads to standardization. Ecma International standardized NLIP as ECMA-430 in December 2025. The relationship between NLIP and HIML described in Section 4 suggests a natural alignment: HIML as the payload specification that NLIP was always designed to carry. A joint working group between the HIML community and the NLIP working group at Ecma International would accelerate both specifications toward interoperability. The Linux Foundation maintains both MCP and A2A. The FIDO Alliance is working on Verifiable Intent. The standards ecosystem is active and looking for the missing pieces.

For The Founders and Practitioners

You do not need to wait for a parser. You do not need to wait for framework support. You can use HIML today.

Take the syntax from Section 6. Write an intent block for the next agent workflow you deploy. Paste it into the system prompt of your agent & any capable large language model will read and follow it. See what changes. See whether your agent makes fewer decisions you would not have made. Then write about what you find. **Post it on X. Publish it on LinkedIn. Share it in the communities where developers and founders talk about building with AI.**

For The Next Generation

There is one more person this paper is written for. Someone young enough that the infrastructure of the agentic web will be built during their lifetime, not inherited from a previous generation, but shaped by their own thinking and their own work. Someone who does not yet have the credentials that academic institutions and standards bodies typically require. Someone who has the rarest possible combination: time to think deeply, freedom from the constraints of an existing paradigm, and the instinct to ask the questions that practitioners who are too close to the current stack have stopped asking.

The people who named the foundational concepts of every previous technology era were not always the most credentialed people in the room. They were often the people who saw the gap most clearly and precisely because they were not yet invested in the systems that created it.

"You do not need permission to contribute to something that does not yet exist. You need a clear argument and the willingness to think in public about something that matters."

The Immediate Next Steps

1. Star the HIML repository on GitHub. Watch it for updates. This is how the community finds each other.
 2. Write a HIML intent block for a real task you are working on today. Use the syntax from Section 6. Paste it into your agent workflow. See what happens.
 3. Post about it publicly on X, LinkedIn, your blog. Tag it #HIML. The community forms around the people who speak first.
 4. If you are a developer, contribute to the reference implementation issue on GitHub. The parser is the first thing the ecosystem needs.
 5. If you are a researcher, pick one open question from Section 9. Write a response to it. Publish it. Start the academic conversation.
 6. If you are at a standards body, reach out. The conversation about where HIML fits in the formal standards ecosystem needs to begin now.
 7. If you disagree with something in this paper, say so. Publicly. Specifically. The strongest version of HIML will be forged in rigorous debate.
-

The AI agent ecosystem is being built right now. The protocols are being written. The standards are being proposed. The infrastructure that will govern how AI systems act on behalf of humans for the next decade is taking shape in GitHub repositories, standards working groups, research papers, and developer communities around the world, today, this week, this year.

The intent layer is the layer closest to humans, the layer that carries human meaning into the systems humans built is the one layer that has not yet been built.

HIML is the proposal to build what is missing. Not someday. Not eventually. Now while the ecosystem is still forming, while the standards are still being written, while the infrastructure is still plastic enough to include a layer that was not in the original design.

The window is open.

The question is who walks through it.

About This Paper

This paper was written as the first formal attempt to name, define, and specify HIML — Human Intent Markup Language as the missing fifth layer of the AI agent infrastructure stack. It is a starting point, not a final specification. This is the Version 1.0. It is open for community review, critique, and contribution.

The Author

This whitepaper was conceived and written by Mohit Rai, who identified the intent preservation gap in multi-agent AI systems and proposed HIML as a structured solution. The author is the Founder and Research Head at Terazap and is not affiliated with any foundation model company, agent framework, or standards body. This work is independent, open, and offered freely to the community by Mohit and Terazap.

How To Cite This Paper

HIML: Human Intent Markup Language — A Proposal for the Missing Layer of the Agentic Web. Version 1.0. 2026.
Available at: <https://github.com/ Terazap/HIML>

License

This specification is published under a Creative Commons Attribution 4.0 International License. You are free to share and adapt the material for any purpose, provided appropriate credit is given and changes are indicated.

Community

GitHub: <https://github.com/ Terazap/HIML>

X / Twitter: #HIML

Discussion: Open issues on the GitHub repository

Reachout to the Author: <https://linkedin.com/in/mohsemohit>

- Soon going to open community portal for collaboration.

"The infrastructure of the agentic web is being built right now. The intent layer is the layer closest to humans and is the one layer that has not yet been built. HIML is the beginning."